



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Universitats i Ocupació

ceedcv

CENTRE ESPECÍFIC
D'EDUCACIÓ A DISTÀNCIA DE
LA COMUNITAT VALENCIANA

Unidad 03 — Hilos y concurrencia

Bloque 01

2º CFGS DAM · Programación de Servicios y Procesos

Sergi García Barea



Lo que veremos en este bloque



De proceso a hilo

Habitaciones de la misma casa



Tu primer hilo

Thread, start y join



Con argumentos

args, kwargs y nombres



Daemon

Fondo que muere con el programa



Timer

Aviso diferido en un hilo

Todo con `threading`: la librería estándar de Python para hilos.

De proceso a hilo

La idea en una frase

“

Un **hilo** es la unidad más pequeña de ejecución: una tarea que vive *dentro* de un proceso y **comparte su memoria** con los demás hilos de ese proceso.

En la unidad pasada lanzaste **procesos**: programas completos con su propia memoria, su propio PID y su propio estado. Ahora bajamos un nivel: un solo proceso puede tener **varios hilos ejecutándose a la vez**, todos con la misma memoria.

Es como pasar de abrir varias casas (procesos) a repartir habitaciones dentro de una sola (hilos).

Un proceso, varios hilos

PROCESO (PID 1000) — UNA SOLA MEMORIA COMPARTIDA

hilo principal (main)
Ejecuta tu programa

thread-1 · worker
Descarga archivos

thread-2 · worker
Escucha la red

thread-3 · daemon
Reloj de fondo

MEMORIA COMPARTIDA · aquí viven las variables que todos los hilos usan

Hilos vs Procesos

La tabla que hay que saberse

Característica	Proceso	Hilo
Memoria	Aislada (cada uno la suya)	Compartida (todos en la misma)
Creación	Lenta (el SO copia recursos)	Rápida
Comunicación	Pipes, sockets, archivos	Variables compartidas
Aislamiento	Alto (uno no afecta a otro)	Bajo (uno puede romper a todos)
Coste	Alto	Bajo



La memoria compartida es a la vez su gran ventaja (comunicación instantánea) y su gran riesgo: si un hilo corrompe una variable, **afecta a todo el proceso**. Ese peligro se llama **condición de carrera**.

Crear y lanzar un hilo

Las tres líneas mágicas

primer_hilo.py

Python

```
import threading
def saludar():
    print("¡Hola desde un hilo!")
hilo = threading.Thread(target=saludar) # 1. crea el hilo
hilo.start()                          # 2. lo lanza
hilo.join()                           # 3. espera a que termine
```

```
$ python primer_hilo.py
¡Hola desde un hilo!
```

Hilos vs Procesos

La analogía de los chefs



Procesos = cocinas de restaurantes diferentes: fogones e ingredientes propios. Si uno se quema, el de al lado ni se entera. Pero abrir un restaurante nuevo es caro y lento.



Hilos = cocineros dentro de la **misma** cocina: comparten encimera, nevera e ingredientes (memoria). Contratar a uno más es barato y rápido.



Si un cocinero tira el aceite caliente, **todos lo sufren**: ese es el bajo aislamiento de los hilos. Ese reparto de la misma nevera hace la comunicación instantánea... y peligrosa si dos tocan el mismo ingrediente a la vez.



En Python, además, los hilos están **limitados por el GIL** para código de cálculo: lo veremos en el bloque 02.

Mini-chequeo: qué es un hilo



Responde en 30 segundos:

1. ¿Dónde vive un hilo: dentro de un proceso o junto a él?
2. ¿Por qué un hilo es más barato de crear que un proceso?
3. ¿Cuál es el precio de compartir memoria entre hilos?



Respuestas:

- **Dentro** de un proceso: varios hilos a la vez, todos con la misma memoria
- Porque **no hay que copiar recursos**: reutiliza la memoria del proceso que ya existe
- El **aislamiento bajo**: si un hilo corrompe una variable compartida, afecta a todos

Tu primer hilo

La idea en una frase

“

Crear un hilo es escribir

`threading.Thread(target=funcion)` , lanzarlo con
`.start()` y esperarlo con `.join()` .

Tres líneas. Eso es todo lo que necesitas para que una función se ejecute en paralelo con el resto del programa. La gracia (y la complicación) está en *cuándo* se ejecuta cada pieza.

1 `threading.Thread(target=saludar)` → **crea** el hilo: existe, pero aún no hace nada (NUEVO)

2 `hilo.start()` → **lo lanza**: pasa a ejecutable y el SO decide cuándo ejecutar `saludar()`

3 `hilo.join()` → **espera** a que el hilo termine antes de seguir

Tu primer hilo

El error típico de principiante



```
Thread(target=saludar())
```

Con paréntesis la función **se ejecuta ahora**, en el hilo principal, y al hilo no le queda nada que hacer: no lanza nada en paralelo.



```
Thread(target=saludar)
```

Sin paréntesis pasas la **referencia** a la función: será el hilo quien la ejecute cuando arranque.



Y recuerda: `start()` solo se puede llamar **una vez** por hilo. Una segunda llamada lanza `RuntimeError: threads can only be started once`. Para repetir el trabajo, crea otro `Thread`.

El hilo principal vs los secundarios

Al ejecutar `python programa.py`, tu código corre en el **hilo principal** (main thread). Todo lo que lanzas con `Thread()` son **hilos secundarios** del mismo proceso.

- 1 El principal crea el hilo secundario con `Thread(target=fn)`
- 2 `start()` lo pone en marcha: ambos avanzan **a la vez**

 El hilo principal **no espera** a los secundarios por arte de magia: hay que decírselo con `join()`. Sin ella, el último `print` del principal puede salir antes que el de los secundarios.

 Matiz importante: el intérprete **sí espera** a los hilos **no-daemon** antes de salir. Lo que pierdes sin `join()` es el **orden** de la salida, no la espera.

join(): esperar a que termine

con_join.py

Python

```
import threading, time
def trabajador(segundos):
    print(f"Trabajando durante {segundos}s...")
    time.sleep(segundos)
    print("Terminado")
h = threading.Thread(target=trabajador, args=(3,))
h.start()
print("Esperando al hilo...")
h.join() # bloquea el principal hasta que el hilo acaba
print("El hilo terminó, continuamos")
```

```
Trabajando durante 3s...
Esperando al hilo...
Terminado
El hilo terminó, continuamos
```

Propiedades de un hilo

propiedades.py

Python

```
hilo = threading.Thread(target=fn, name="hilo-1")
hilo.start()
hilo.join()
print(hilo.name)      # "hilo-1"
print(hilo.ident)     # ID numérico del hilo
print(hilo.daemon)    # True/False
print(hilo.is_alive())
```

Propiedad	Qué devuelve
.name	Nombre del hilo (por defecto "Thread-1")
.ident	ID numérico único mientras vive
.daemon	True si es hilo daemon
.is_alive()	True si aún se está ejecutando



Desde dentro:

`threading.current_thread()` devuelve el objeto Thread del hilo que se está ejecutando.

Hilos con argumentos

La idea en una frase

“

Para que un hilo trabaje con datos propios le pasas argumentos en `args=` (tupla) o `kwargs=` (diccionario), y para distinguirlo le pones nombre con `.name`.

Sin argumentos, todos los hilos harían exactamente lo mismo. Con `args` y `kwargs`, cada hilo hace su versión del trabajo. Y con `.name`, sabes en cada momento *quién* hace qué.

Pasar argumentos: args y kwargs

args_kwargs.py

Python

```
def trabajar(nombre, tarea):  
    print(f"{nombre}: {tarea}")  
# tupla posicional  
h1 = threading.Thread(  
    target=trabajar,  
    args=("Ana", "lavar platos"))  
# diccionario con nombre  
h2 = threading.Thread(  
    target=trabajar,  
    kwargs={"nombre": "Bob",  
            "tarea": "fregar suelo"})
```

args

Tupla con los posicionales, en el orden que espera la función

kwargs

Diccionario con los argumentos por nombre



Truco de la coma: una tupla de un elemento lleva coma final: `args=("Ana",)`. Sin ella, Python itera el string y pasa `'A'`, `'n'`, `'a'` como tres argumentos → `TypeError`.

Nombrar hilos y lanzar varios a la vez

varios_hilos.py

Python

```
def tarea(n):
    nombre = threading.current_thread().name
    print(f"{nombre} → tarea {n}")
hilos = [
    threading.Thread(
        target=tarea, args=(i,),
        name="hilo-" + str(i))
    for i in range(1, 5)
]
for h in hilos: h.start()
for h in hilos: h.join()
print("Todas las tareas terminadas")
```



`name="hilo-" + str(i)` da un **nombre único** a cada hilo, legible desde dentro con `current_thread().name`



Primer bucle: **lanza todos**. Segundo bucle: **espera todos**

```
hilo-1 → tarea 1
hilo-3 → tarea 3
hilo-2 → tarea 2
hilo-4 → tarea 4
Todas las tareas terminadas
```



El orden de los `print` lo decide el **scheduler del SO**: no hay garantía. El mensaje final, en cambio, **siempre** es el último: lo avalan los `join()`.

Mini-chequeo: argumentos



Responde sin mirar:

1. ¿Qué le pasas a `args=` ? ¿Y a `kwargs=` ?
2. ¿Por qué hace falta la coma en `args=("Ana",)` ?
3. ¿Cómo sabes, dentro de la función del hilo, qué hilo te está ejecutando?



Respuestas:

- A `args=` una **tupla** de posicionales; a `kwargs=` un **diccionario** de argumentos con nombre
- Porque `("Ana")` es solo un string: con la coma, Python entiende que es una **tupla de un elemento**
- Con `threading.current_thread().name` : el objeto Thread que te está ejecutando, y su nombre

Hilos daemon

La idea en una frase

“

Un hilo **daemon** se ejecuta en segundo plano y **se mata automáticamente** cuando el programa principal termina. Los no-daemon, en cambio, impiden que el programa salga hasta terminar.

Imagina una alarma de incendios: la quieres encendida mientras el edificio está vivo, pero si el edificio desaparece, la alarma no tiene sentido y muere con él. Esa es la filosofía del daemon: tareas auxiliares de fondo (monitorizar, limpiar, un reloj, un heartbeat).

Daemon en acción

El mismo trabajador, con y sin daemon

SIN daemon — el programa espera

```
h1 =  
threading.Thread(target=trabajador)  
  
h1.start() · h1.join()
```

El hilo completa sus 5 pasos y **después** termina el programa.

CON daemon — el hilo muere al salir

```
h2 = threading.Thread(target=trabajador,  
daemon=True)  
  
h2.start() · time.sleep(1.2)
```

El daemon solo llegó al paso 3: el principal terminó y **lo mató en seco**.

 Aquí no hay `join()` en la versión daemon: no queremos esperarlo, justo lo contrario. Para esperas limitadas existe `h.join(timeout=3)` : espera como mucho 3 segundos y sigue.

Hilos daemon

Cada uno tiene su sitio

✓ Hilo normal + join()

- Tareas **críticas** que deben completarse sí o sí
- Escribir datos, cerrar una operación, un cálculo que no puede quedarse a medias

😈 Hilo daemon

- Servicios de fondo **prescindibles**: monitorización, limpieza, heartbeat, un reloj
- Da igual que se corten: el programa puede salir sin ellos

⚠ Un daemon cortado **en seco** puede dejar datos a medias: si un archivo debe quedar bien escrito, no lo escribas desde un daemon.

Timer: avisos diferidos

La idea en una frase

“

`threading.Timer(retardo, funcion)` ejecuta `funcion` **una sola vez** después de `retardo` segundos, sin bloquear el resto del programa.

timer.py

Python

```
def aviso():  
    print("🕒 ¡Tiempo cumplido!")  
t = threading.Timer(5.0, aviso)  
t.start()  
time.sleep(2)  
t.cancel() # cancela antes de disparar
```

- 1 **No bloquea:** el Timer es un hilo más
- 2 Dispara la función **una sola vez**
- 3 `t.cancel()` anula el disparo si aún no ha ocurrido

Usos típicos del Timer



Aviso

"¡Despierta!" a los
3s



Recordatorio

Mensaje tras un
rato



Timeout

Cortar lo que
tarda



Sesión

Logout por
inactividad



Limpieza

Borrar temporales



El Timer **hereda el daemon** del hilo que lo crea: por defecto el intérprete **espera al disparo** antes de salir. Para no alargar el programa: `t.daemon = True` antes de `start()`.

Mini-chequeo: daemons y Timer



Responde en 30 segundos:

1. ¿Un hilo daemon impide que el programa termine?
2. ¿Qué hace `join(timeout=2)` si el hilo tarda 5 segundos?
3. ¿Cuántas veces ejecuta el Timer la función?



Respuestas:

- **No:** se mata al terminar el principal. Quien impide la salida es el hilo **no daemon**
- Espera 2 segundos como máximo y **continúa**, aunque el hilo siga vivo (`is_alive()` lo confirmaría)
- **Una sola vez**, tras el retardo. Para repetir, la función debe reprogramarse a sí misma

Resumen del bloque 01



Hilo

Vive en un
proceso; memoria
compartida, sin
aislamiento



Ciclo básico

Thread · start() ·
join()



args / kwargs

Tupla y
diccionario;
.name



Daemon

Fondo
prescindible;
crítico → join()



Timer

Un disparo
diferido



Bloque 02: **GIL**, estados del hilo y taller de trazas.

